

Phases of Testing in Generative AI

This module explains how **Generative AI enhances each testing phase** and supports modern testing practices.

Phases of Testing

Traditional Testing Phases:

1. Requirement Analysis
2. Test Planning
3. Test Design
4. Test Environment Setup
5. Test Execution
6. Defect Reporting
7. Test Closure

Generative AI adds intelligence across **all phases**.

Applications of Generative AI in the Software Testing Phase

GenAI helps to:

- Generate test cases
 - Create test data
 - Optimize regression tests
 - Predict failures
 - Suggest scenarios
 - Automate documentation
-

Automated Test Case Generation

GenAI converts:

- ✓ Requirements → Test cases
- ✓ User stories → Scenarios
- ✓ Code → Unit tests

Example Prompt

Generate test cases for password reset feature.

Demo: Automating Test Case Generation Using Copilot

Step 1

Open file `test_login.py`

Step 2

Type comment:

```
# Generate pytest test cases for login function
```

Step 3

Copilot suggests multiple test cases.

Test Data Generation and Augmentation

GenAI creates:

- Valid data
- Invalid data
- Boundary values

Example

```
# Generate test data for email field
```

Scenario-Based Testing

Testing real-world usage flows.

Example:

User registers → logs in → adds product → checkout.

Scenario-Based Testing: GenAI's Contribution

GenAI:

- Creates realistic flows
 - Identifies alternate paths
 - Suggests edge cases
-

Demo: Creating and Executing Scenario-Based Testing Using GitHub Copilot

Prompt

```
# Create scenario tests for shopping cart
```

Copilot generates multi-step test cases.

Regression Testing Optimization

GenAI:

- Selects impacted tests
 - Removes redundant tests
 - Prioritizes tests
-

Regression Testing Optimization: Benefits

- Faster regression cycles
 - Reduced cost
 - Higher coverage
-

Failure Prediction and Analysis

GenAI analyzes historical defects to:

- Predict risky modules
 - Suggest preventive tests
-

Failure Prediction and Analysis: Benefits

- Early bug detection
 - Reduced production failures
-

Dynamic Test Case Adjustment

GenAI updates tests automatically when code changes.

Exploratory Testing Support

GenAI suggests:

- Areas to explore
 - Inputs to try
 - Suspicious patterns
-

Integration with Testing Tools

GenAI integrates with:

- Selenium
 - Playwright
 - Cypress
 - PyTest
 - JUnit
 - GitHub Actions
-

Integration with Testing Tools: Example

```
name: Run Tests

on: [push]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Run PyTest
        run: pytest
```

Copilot can generate this YAML.

DEMO: Creating Dynamic Test Cases and Integrating GitHub Actions with GitHub Copilot

Step 1

Create workflow file `.github/workflows/tests.yml`

Step 2

Type:

```
# Create CI pipeline to run python tests
```

Copilot suggests pipeline.

Limitations

- Needs human review
 - Might miss business rules
-

Quick Check

Q1. What is scenario-based testing?

Testing real user flows

Q2. True/False

GenAI can generate test data.

True

Q3. Which tool can run tests in CI?

GitHub Actions

Summary

Generative AI enhances testing by:

Automating test creation

Improving coverage

Optimizing regression

But **human expertise remains essential.**
